

ENTREGABLE 3 - PROYECTO



David Esteban Beltrán Gómez

Juan Felipe Gómez López

Ana María Romero Cuadrado

Facultad de ingeniería

Pontificia Universidad Javeriana

Estructuras de Datos

28 de noviembre de 2025

Bogotá D.C., Colombia

Introducción

Este documento describe el diseño del sistema para el Componente 3 del proyecto del curso Estructuras de Datos, además de describir los errores y sus soluciones de los anteriores componentes 1 y 2. El sistema permite cargar, manipular, guardar secuencias genéticas en formato FASTA, codificar y decodificar archivos binarios. Lo más importante que se revisara en esta entrega son los comandos ruta más corta entre una base origen y una base destino, y ruta remota para encontrar la base más lejana al origen que coincidan siendo la misma base (misma letra) utilizando estructuras no lineales. Como bien se sabe, el usuario debe digitar por consola los comandos disponibles y el sistema deberá proporcionar la respectiva salida. Al final del documento se pueden visualizar todas las pruebas que se hicieron para confirmar el funcionamiento del componente tres.

Acta de cada entrega

Entrega 1

Documento de diseño

1.

“Especificar exactamente qué tipo de estructura lineal es una colección.”

En cada uno de los TADs se especifica que tipo de estructura lineal se va a utilizar dependiendo de las funcionalidades que esta va a cumplir.

2.

“Mantener el mismo formato en la presentación de los TADs.”

Se implemento la misma plantilla para cada uno de los TAD, implementando la misma tipografía y características en cada uno para mantener consistencia entre cada uno de los TADs.

Plan de pruebas

1.

“Sólo era necesario realizar el plan de pruebas para el comando enmascarar.”

Sólo se realizó el plan de pruebas únicamente para el comando “decodificar” como se especifica en el enunciado para la entrega 2. No se realizaron plan de pruebas para los demás comandos.

Código Fuente

1.

“No es necesario que salga “[OK] comando...” cada vez que se puede ejecutar el comando. Eso servía para la entrega 0, pero ahora sólo es necesario que salgan los mensajes definidos en el enunciado.”

Se elimina el **cout** de confirmación, dejando en el código únicamente la información relevante e imprimiendo los elementos necesarios según los requisitos que hay en el enunciado. De esta manera se proyecta en pantalla únicamente la información importante y no elementos que pueden llevar a distorsionar la información.

2. Comando cargar

“El comando no funciona con archivos largos, con yeast.fa me generó violación de segmento”

Este error era ocasionado por el uso de regex, el cual permitía solo los caracteres establecidos. Esto se tuvo que cambiar ya que al manipular archivos tan largos regex llenaba su memoria y por eso ocurría la violación. Por lo tanto, se cambió a una validación por cada carácter de la cadena con un **for** al encontrar un carácter diferente a los que se establecieron como permitidos, retorna false y esa secuencia no la guarda en el sistema.

3. Comando cargar

“Aunque el comando sólo está cargando en memoria las secuencias que sólo tienen símbolos válidos, no presenta un mensaje indicando al menos cuáles secuencias no pudieron ser cargadas.”

Se implementó list<Secuencia> invalidas para almacenar las secuencias que al realizar la verificación no cumplan con las características necesarias. Finalmente se imprime en pantalla la descripción de aquellas las secuencias que tienen algún carácter invalido.

4. Comando histograma

“Como sugerencia, no incluir las bases que tengan conteo igual a 0”

Se realiza una verificación de cuáles son las bases con una frecuencia mayor a 0 y de esta manera se imprimen en pantalla únicamente las bases que aparecen una vez o más. Esto permite visualizar únicamente las bases presentes y no una lista de todas las bases con varias de estas en 0. Lo cual dificultaba la comprensión del histograma

Entrega 2

Documento de diseño

1.

“En la descripción del TAD NodoHuffman, hace falta incluir como dato mínimo el hijo derecho”

Como se puede visualizar en la siguiente sección de los TADs, ya se hizo la inclusión del hijo derecho en NodoHuffman, esto para que tenga coherencia con lo realizado en la implementación del código fuente.

2.

“El documento es incremental, lo que quiere decir que simplemente le agrego las partes nuevas en los puntos que sea necesario: TADs nuevos en la descripción de los TADs y en el diagrama de relación, esquemáticos nuevos junto con los anteriores, plan de pruebas nuevo junto con el anterior. No separar las entregas por partes por favor.”

En esta última entrega se hizo la implementación de todas las entregas anteriores, así se visualizan mejor los cambios realizados y no pierde conocimiento del todo el trabajo realizado en este proyecto.

3.

“En el diagrama de relación, el árbol se utiliza de forma indirecta desde el genoma (no como un dato mínimo sino dentro de una función), por lo que la conexión debe ajustarse para indicar "usa", junto a la cardinalidad de la conexión.”

Se corrigió el diagrama para que todo tuviese sentido y conexión, así no queda algún TAD “flotando”, la cardinalidad se hizo de 1 a 1, ya que en este caso solo se construye un árbol con todas las secuencias que están cargadas en el sistema, así que cada vez que el sistema carga un nuevo archivo, se “borra” el anterior árbol y crea uno nuevo, pero siempre será solo 1.

Código Fuente

1.

“En el archivo binario, la información no se está almacenando en el orden esperado. Todos los nombres de las secuencias deberían estar juntos inicialmente, y luego deberían estar todas las codificaciones de las secuencias.”

Ya se hizo el pequeño cambio en el código, anteriormente se escribía y leía las descripciones en un ciclo junto con la longitud del código, ancho de línea y el código binario. Esto hacía que se escribiera parte por parte. Así que se dividió este proceso en dos ciclos **for** aparte, uno para escribir o leer solo la información de las descripciones y al terminar este se escribe o lee toda la información necesaria del código de cada secuencia.

Descripción de TAD utilizados

Entrega 1

TAD Secuencia

Conjunto mínimo de datos

- descripción, cadena de caracteres, cadena que identifica la secuencia, comenzando con ">" en el formato original.
- bases, cadena de caracteres, cadena que contiene las bases nitrogenadas (A, C, G, T, U, R, Y, K, M, S, W, B, D, H, V, N, X, -) representando la secuencia genética.
- anchoLinea, entero, longitud constante de las líneas en las que se divide la secuencia al guardarla, determinada por la primera línea de bases.

Comportamiento

- Secuencia(), constructor por defecto
- Secuencia(descripción), constructor con la descripción de la secuencia
- Secuencia(descripción, bases, ancho), constructor de una secuencia con la información completa.
- setDescription(descripción), fija la descripción enviada por parámetro
- getDescription(), devuelve la descripción de la secuencia.
- agregarLinea(linea), agrega una línea de bases a la secuencia.
- getBases(), Obtiene las bases de la secuencia.
- getAnchoLinea(), Obtiene el ancho de la línea

- `secuenciaCompleta()`, Devuelve verdadero o falso dependiendo si la secuencia tiene “-” o no.
- `contarBases(bases, conteoBase)`, devuelve la cantidad de bases contadas incluyendo las inválidas y actualiza la frecuencia de cada base en la secuencia.
- `reemplazarSubsecuencia(subsecuencia, posición)`, reemplaza una subcadena de la secuencia y la reemplaza por un carácter.
- `contarSubsecuencia(subsecuencia)`, cuenta cuantas veces existe una subsecuencia en las secuencias almacenadas en memoria

TAD Genoma

Conjunto mínimo de datos

- Secuencias, arreglo dinámico de objetos de tipo Secuencia, contiene todas las secuencias genéticas cargadas en memoria.

Comportamiento

(Creados en primera entrega)

- `cargarDesdeArchivo(nombreArchivo)`, lee un archivo en formato FASTA y agrega todas las secuencias encontradas al conjunto.
- `guardarArchivo(nombreArchivo)`, escribe las secuencias almacenadas en memoria en un archivo FASTA, respetando el formato y el ancho de línea.
- `listar_secuencias()`, muestra en pantalla todas las descripciones de las secuencias almacenadas en memoria.
- `histograma(descripcion)`, genera y muestra un histograma con la frecuencia de bases nitrogenadas en la secuencia indicada.
- `esSubsecuencia(sub, secuencia)`, verifica si una cadena es subsecuencia de otra secuencia almacenada.
- `enmascarar(descripcion, subsecuencia, caracter)`, busca una subsecuencia dentro de la secuencia indicada y la reemplaza con un carácter de enmascaramiento (ej: N o *).
- `buscarSecuencia(descripcion)`, devuelve la secuencia correspondiente a una descripción dada, si existe.

(Creados en segunda entrega)

- `calcularFrecuencia(frecuencias, bases)`, calcula la frecuencia de cada base genética y llena los vectores dados.
- `leerBitsDesempaquetados(entrada, bytesToRead)`, lee una cantidad de bytes del archivo y devuelve su representación en bits.
- `escribirBitsEmpaquetados(salida, bitString)`, escribe una cadena de bits en un archivo empaquetándolos en bytes.
- `codificar(nombreArchivo)`, codifica todas las secuencias en formato binario comprimido y las guarda en el archivo indicado.
- `decodificar(nombreArchivo)`, lee un archivo codificado y reconstruye las secuencias originales en memoria.

(Creados en tercera entrega)

- ruta_mas_corta(descripcion, i, j, x, y), calcula la ruta más corta entre dos posiciones (i, j) y (x, y) dentro del grafo de la secuencia indicada.
- base_remota(descripcion, i, j), encuentra dentro de la secuencia indicada la base del mismo tipo que se encuentre a mayor distancia (dependiendo del costo).

Entrega 2

TAD NodoHuffman

Conjunto mínimo de datos

- simbolo, caracter, representa el dato de la base genética que tiene el nodo Huffman (únicamente los nodos hoja tendrán este dato).
- frecuencia, entero grande, permite almacenar la frecuencia de aparición de una base genética en todas las secuencias cargadas en memoria.
- izquierda, apuntador de tipo NodoHuffman, permite almacenar el hijo izquierdo del nodo del árbol.
- derecha, apuntador de tipo NodoHuffman, permite almacenar el hijo derecho del nodo del árbol.

Comportamiento

- NodoHuffman (simbolo, frecuencia), constructor para los nodos hoja los cuales no tienen hijos y tendrán únicamente el dato de la frecuencia y el simbolo (base)
- NodoHuffman (frecuencia, izquierda, derecha), constructor para los nodos intermedios incluida la raíz, como parámetros recibe el dato de frecuencia y los hijos.
- esHoja(), Esta función permite saber si un nodo es hoja mirando si tiene hijos o no y devuelve un booleano.

TAD ArbolHuffman

Conjunto mínimo de datos

- raiz, apuntador de tipo NodoHuffman, permite tener referencia de la raíz y por ende el acceso al árbol.

Comportamiento

- ArbolHuffman(), función genérica para la construcción de árbol,
- fijarRaiz (simbolos, frecuencias), es la función que permite construir el árbol de Huffman dada una colección de símbolos y una colección de frecuencias en la que cada posición i de cada colección representa un símbolo y su frecuencia.
- getRaiz (), esta función devuelve un apuntador a la raíz del árbol.
- generarCodigos (simbolos, codigos), esta función es la encargada de generar los códigos para cada base, recibe como parámetro una colección de símbolos y una colección de códigos donde se almacenarán los códigos de cada base genética.

- generarCodigosRec (simbolos, codigo, simbolos, codigos), función recurrente que permite recorrer cada camino del árbol hacia los nodos hojas e ir concatenando los caracteres 0 o 1 para obtener el código final de la base genética.
- insertarOrdenado (nodos, nuevo), función que permite insertar un nuevo nodo originado en el vector de nodos que se recibe por parámetro sin alterar la propiedad de orden que tiene el vector.
- liberarArbol (nodo), función que permite vaciar el árbol y sus nodos
- decodificar (codigoBinario, lenght), función que permite decodificar una secuencia leída desde un archivo binario.

Entrega 3

TAD NodoGrafo

Conjunto mínimo de datos

- fila, entero, indica la posición en filas de la base dentro de la matriz.
- col, entero, indica la posición en columnas de la base dentro de la matriz.
- base, carácter, almacena la base genética correspondiente a la posición.

Comportamiento

- NodoGrafo(), constructor por defecto, crea un nodo vacío.
- NodoGrafo(fila, col, base), constructor que inicializa un nodo con su posición y la base correspondiente.
- NodoGrafo(fila, col), constructor que inicializa un nodo solo con su posición ya que la base estaría vacía.
- getBase(), devuelve el carácter que representa la base genética del nodo.
- getFila(), devuelve la fila del nodo en la matriz.
- getColumna(), devuelve la columna del nodo en la matriz.

TAD GrafoBase

Conjunto mínimo de datos

- matriz, arreglo bidimensional (vector de vectores) de NodoGrafo, almacena todos los nodos del grafo representando la matriz de bases.
- filas, entero, número de filas de la matriz.
- cols, entero, número de columnas de la matriz.

Comportamiento

- GrafoBase(bases, cols, filas), constructor que recibe una cadena con las bases y las dimensiones de la matriz, creando el grafo completo.

- o `posicionValida(i, j)`, devuelve verdadero si la posición (i,j) existe en la matriz, falso en caso contrario.
- o `getBasePosicion(i, j)`, devuelve la base en la posición (i,j) de la matriz.
- o `vecinos(i, j)`, devuelve un vector de pares (NodoGrafo, double) con los nodos vecinos y el peso de la arista correspondiente.
- o `dijkstra(i, j, x, y, costoFinal)`, calcula la ruta más corta desde la posición (i,j) hasta (x,y), devuelve un vector de nodos en la ruta y asigna el costo total a `costoFinal`.
- o `peso(a, b)`, devuelve el peso (costo) de ir de la base *a* a la base *b*.
- o `baseMasRemota(origen)`, devuelve un par: el nodo que contiene la misma base más lejana del nodo origen y el vector con la ruta hacia él.

Diagrama de clases

Entrega 1



Figura 1. Diagrama de TADs – entrega 1

Entrega 2

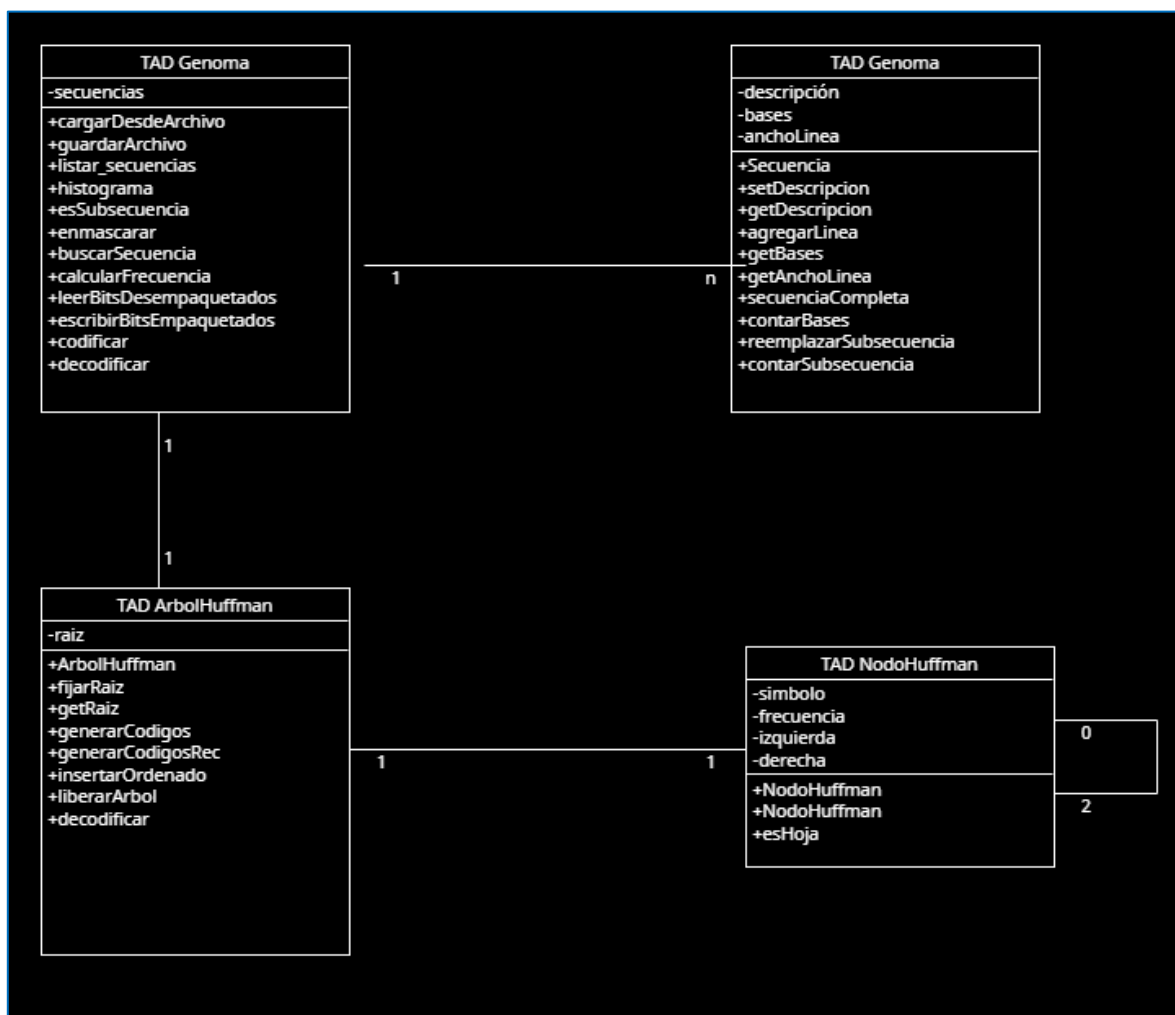


Figura 2. Diagrama de TADs – entrega 1 y 2

Entrega 3

Diagrama final completo (componentes 1,2 y 3)

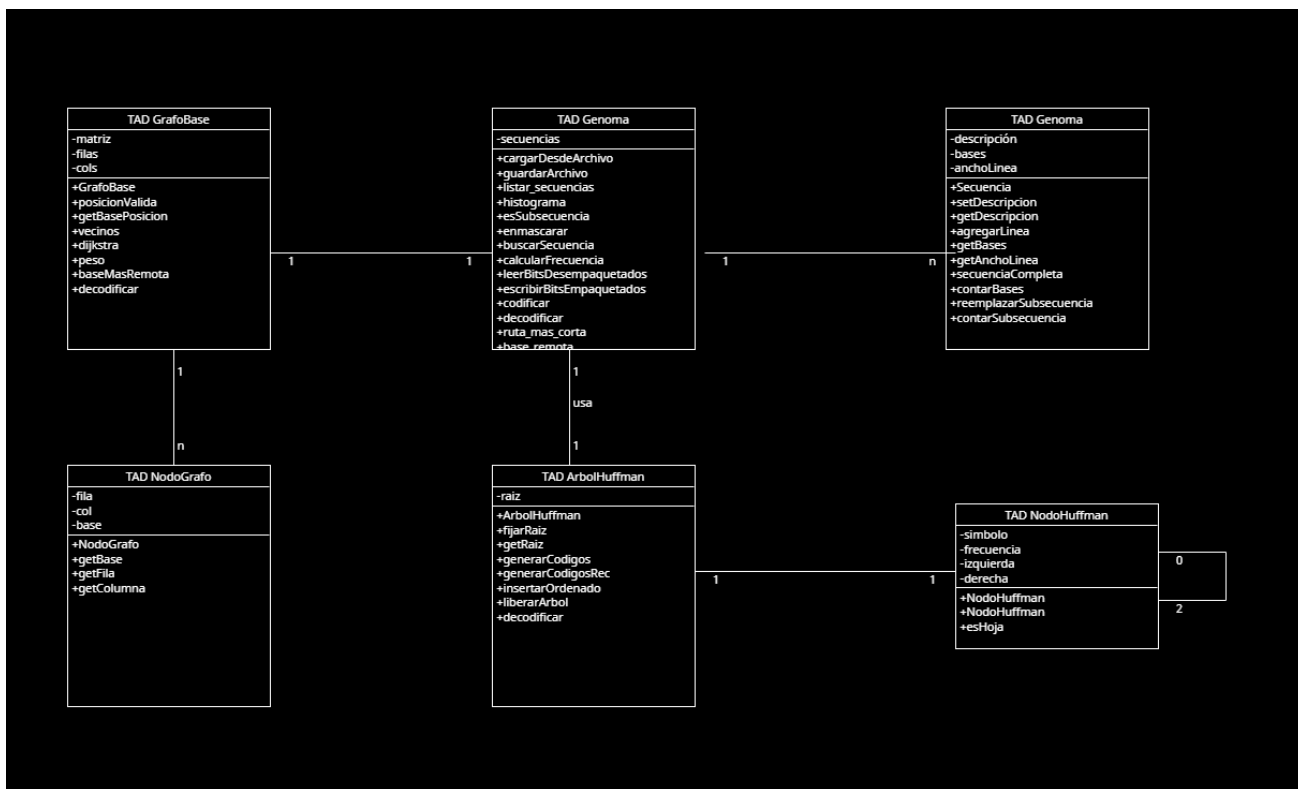


Figura 3. Diagrama de TADs – entrega 1, 2, 3

Diagramas o esquemáticos

Aquí se va a enseñar el proceso que sigue cada función para realizar su tarea, si quiere visualizar mejor los de esta entrega puede dirigirse a la parte de entrega 3.

Entrega 1

Función cargarArchivo:

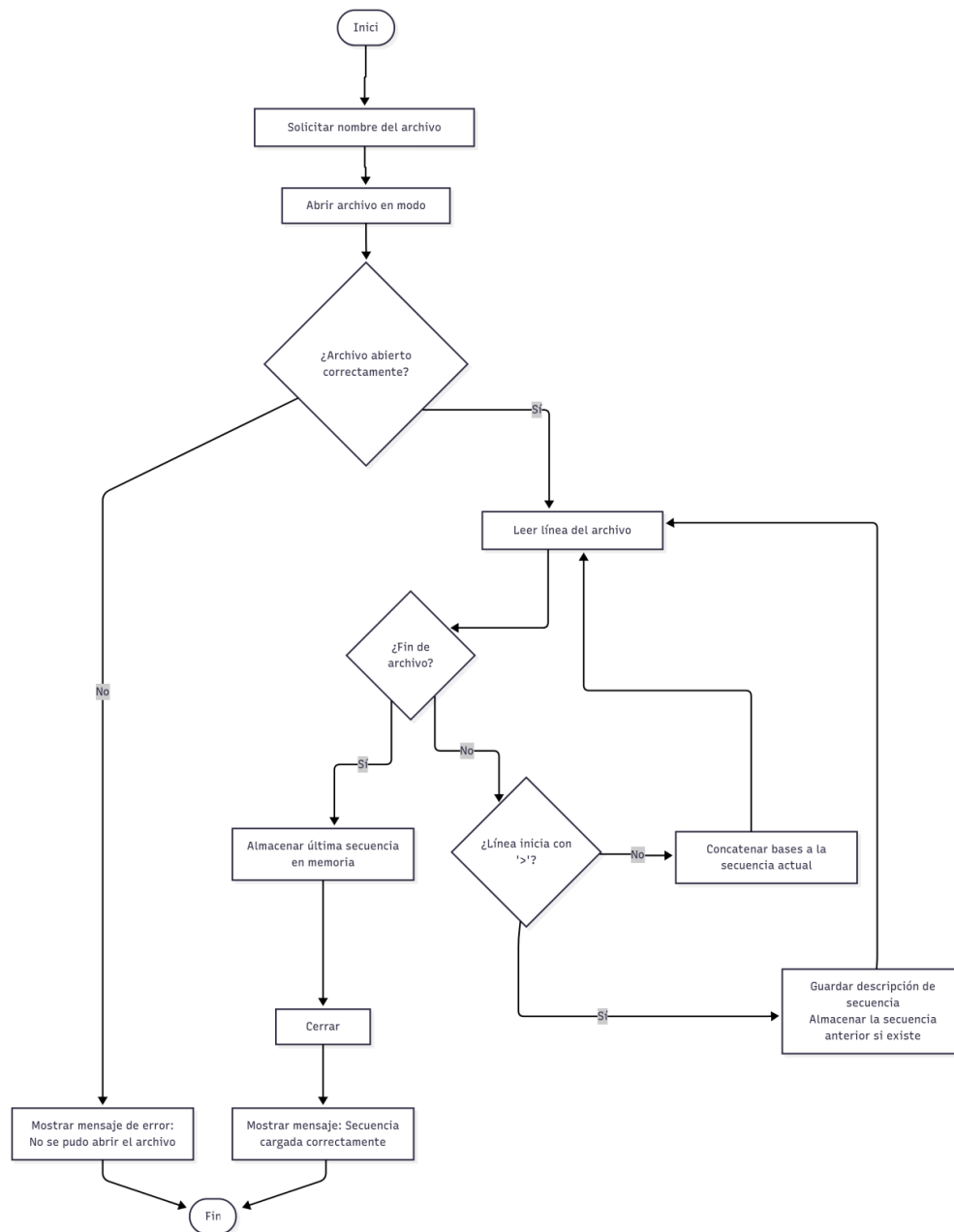


Figura 4. Diagrama de flujo – función cargar archivo

Descripción:

Lee un archivo en formato FASTA: obtiene la descripción de cada secuencia (líneas que empiezan con >) y concatena las bases que la siguen. Al final, guarda todas las secuencias en memoria.

Función guardarArchivo:

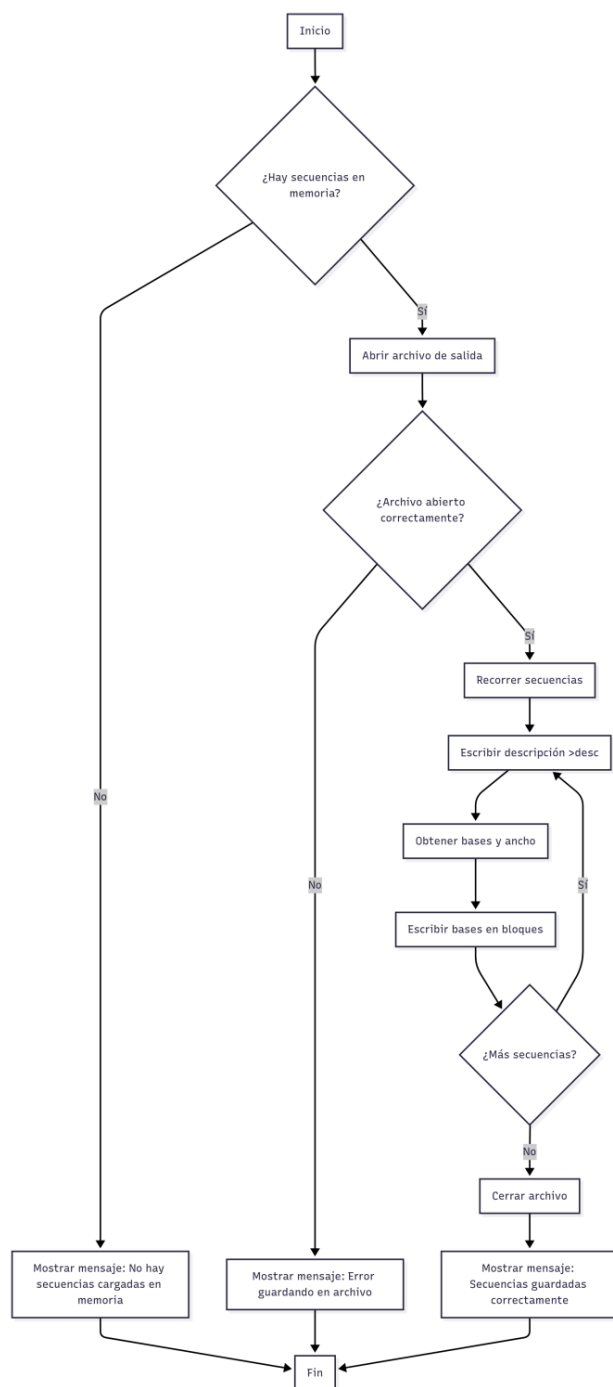


Figura 5. Diagrama de flujo – función guardarArchivo

Descripción:

Escribe en un archivo todas las secuencias almacenadas en memoria, incluyendo su descripción y sus bases (estas en bloques de 80 caracteres por línea).

Función esSubsecuencia:

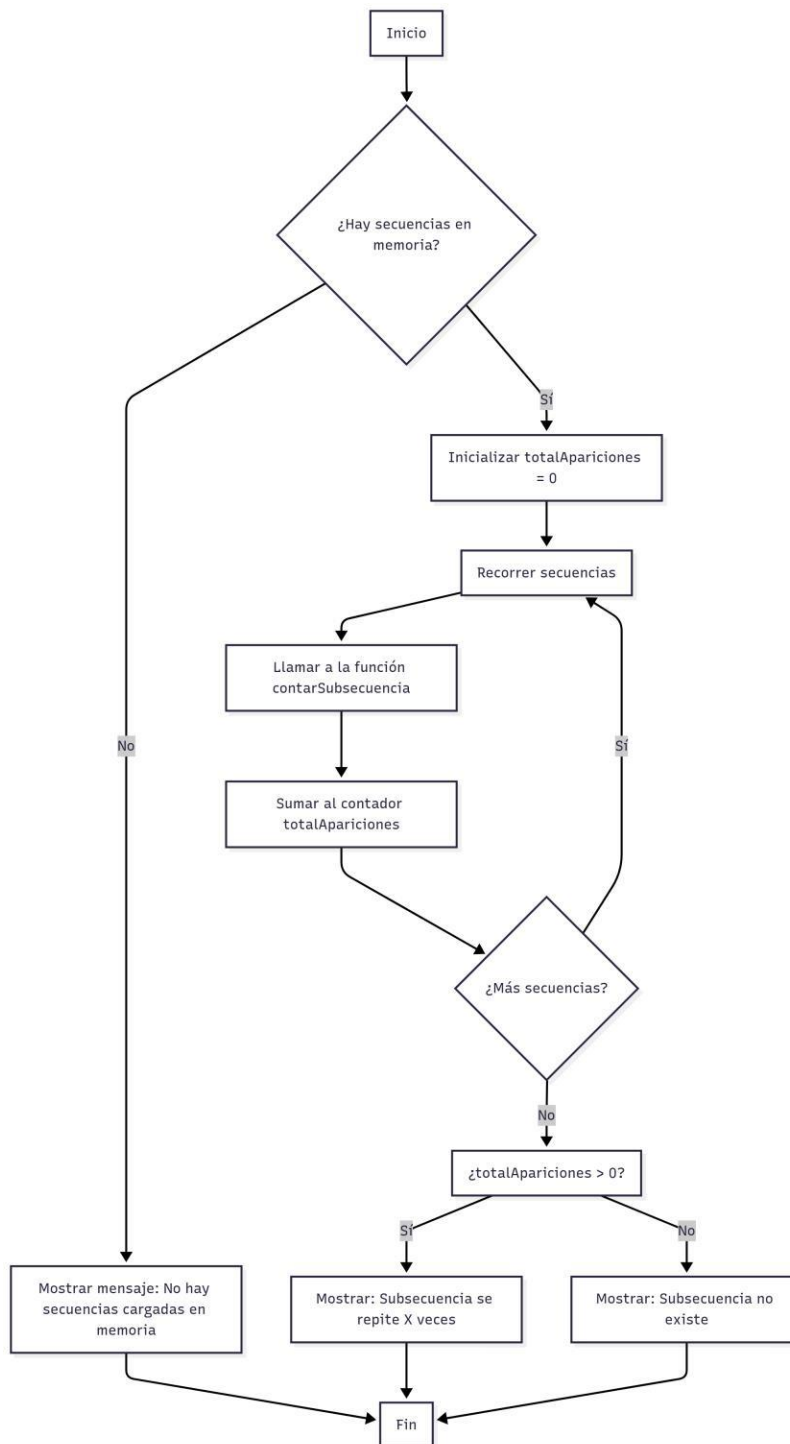


Figura 6. Diagrama de flujo – función esSubsecuencia

Descripción:

Verifica si una secuencia está contenida (como subsecuencia) dentro de otra. Recorre las bases en orden y confirma si se cumplen todas.

Función Enmascarar:

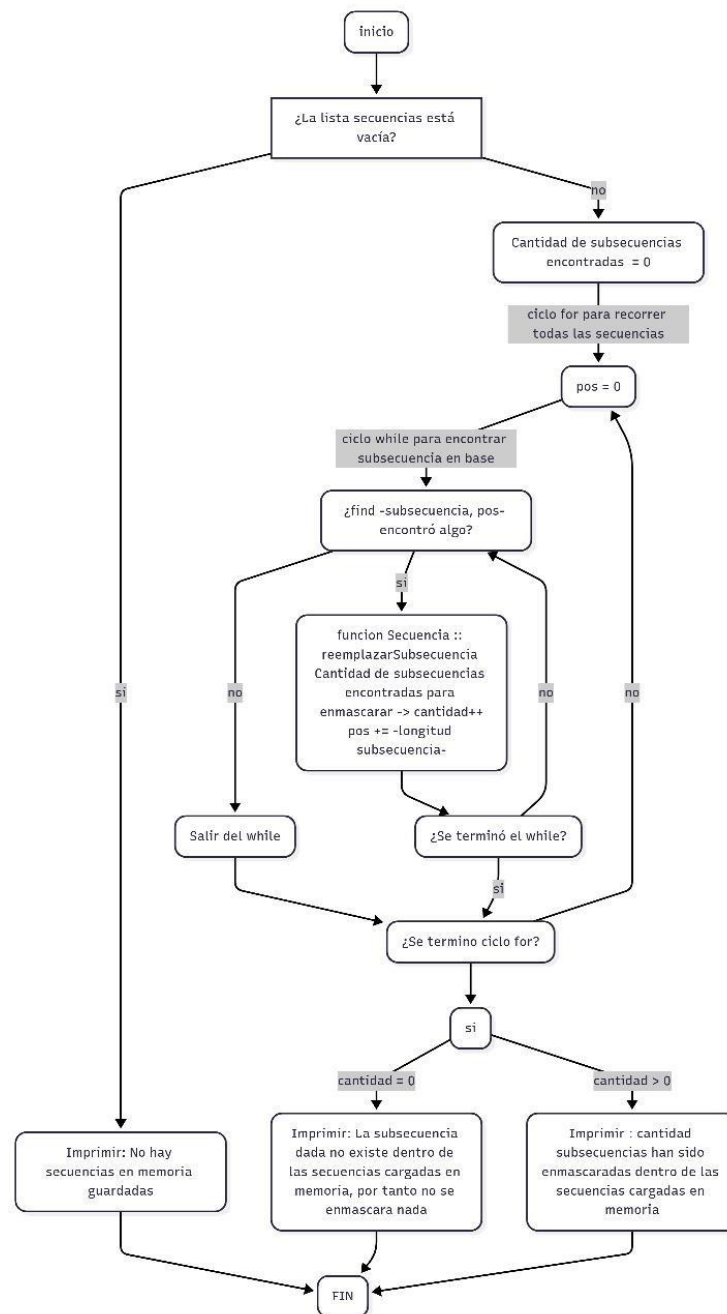


Figura 7. Diagrama de flujo – función Enmascarar

Descripción:

Recibe una secuencia y reemplaza (enmascara) todas las apariciones de una subsecuencia dentro de ella por el carácter 'X'. Sirve para ocultar patrones específicos en la secuencia.

Función listarSecuencias:

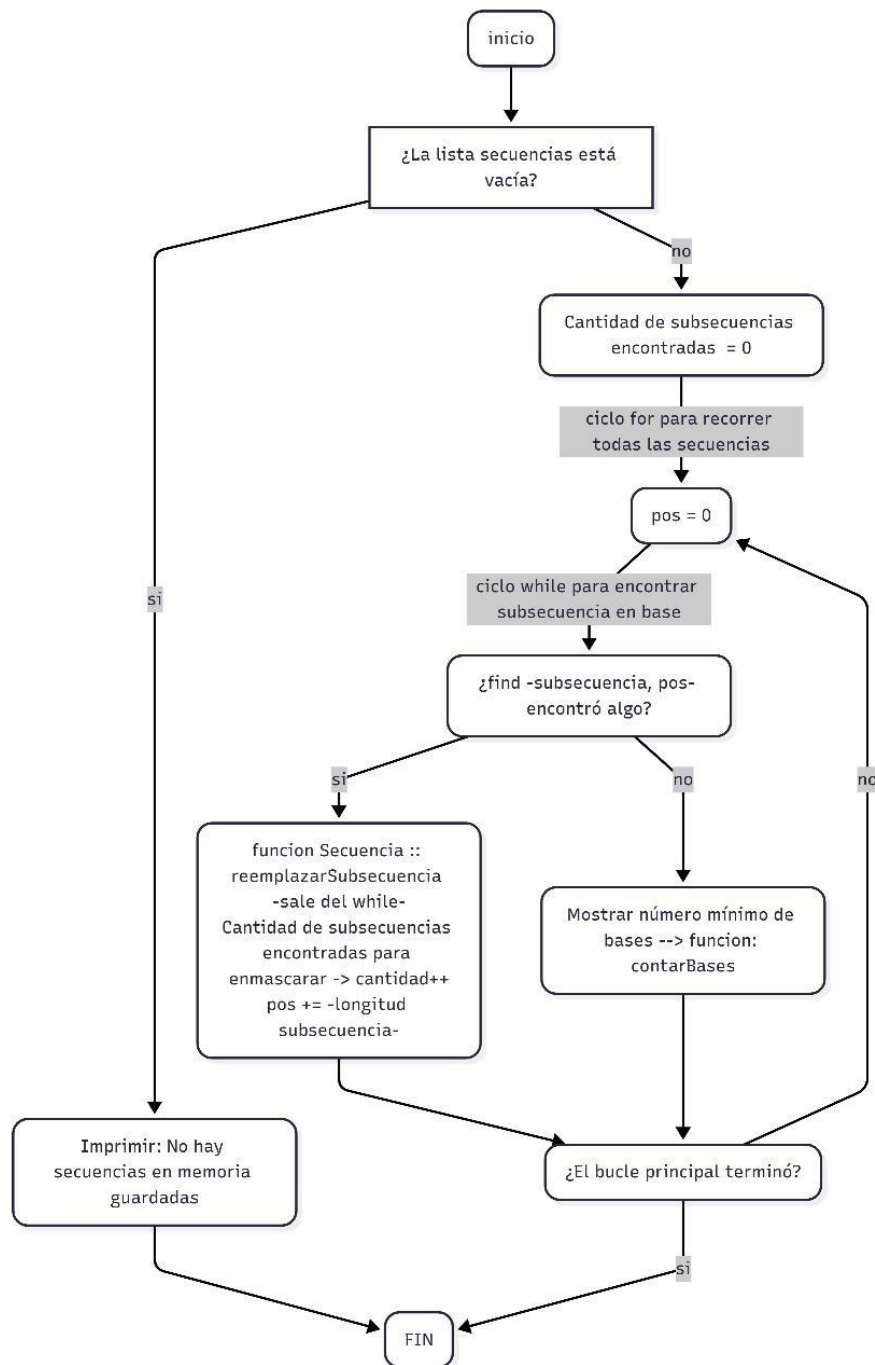


Figura 8. Diagrama de flujo – función listarSecuencias

Descripción:

Muestra en pantalla todas las secuencias cargadas en memoria junto con su descripción, para que el usuario pueda revisarlas.

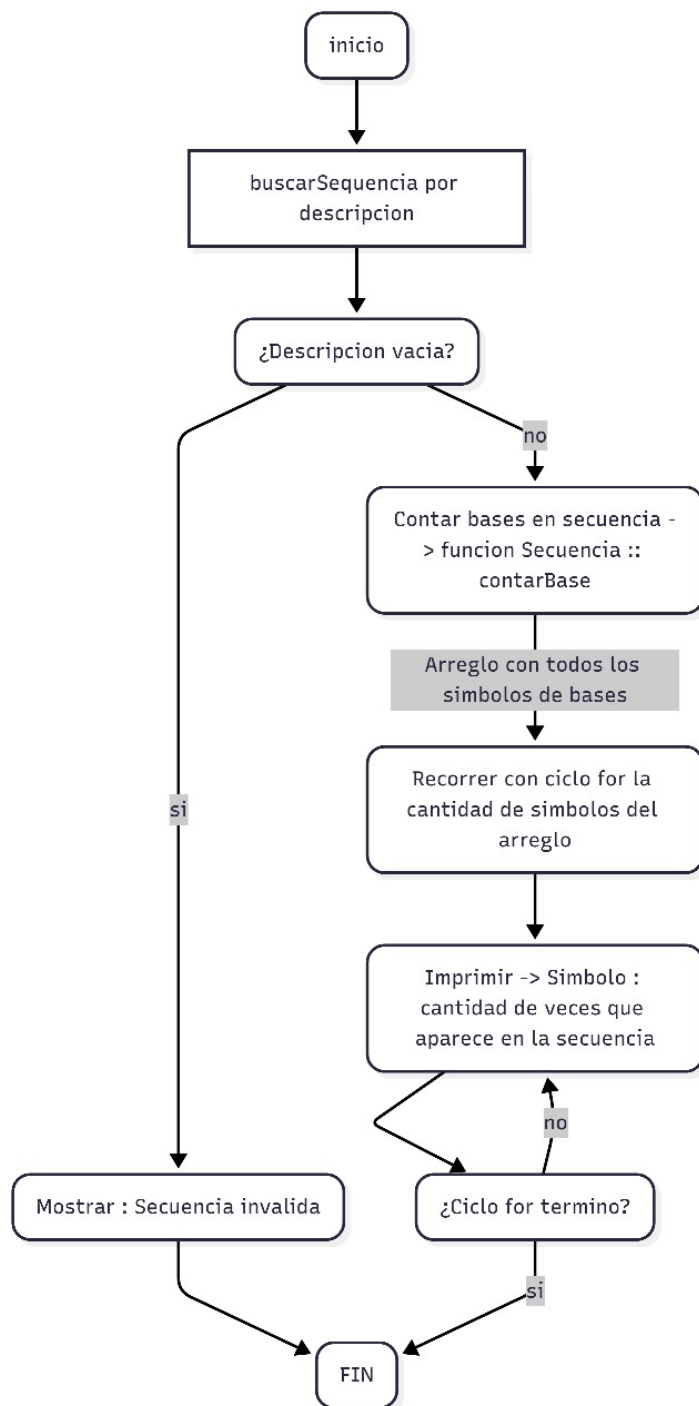
Función Histograma:

Figura 9. Diagrama de flujo – función Histograma

Descripción:

Cuenta la frecuencia de cada base nitrogenada en una secuencia y muestra los resultados en forma de histograma de texto (por ejemplo, usando barras con asteriscos o números).

Función buscarSecuencia:

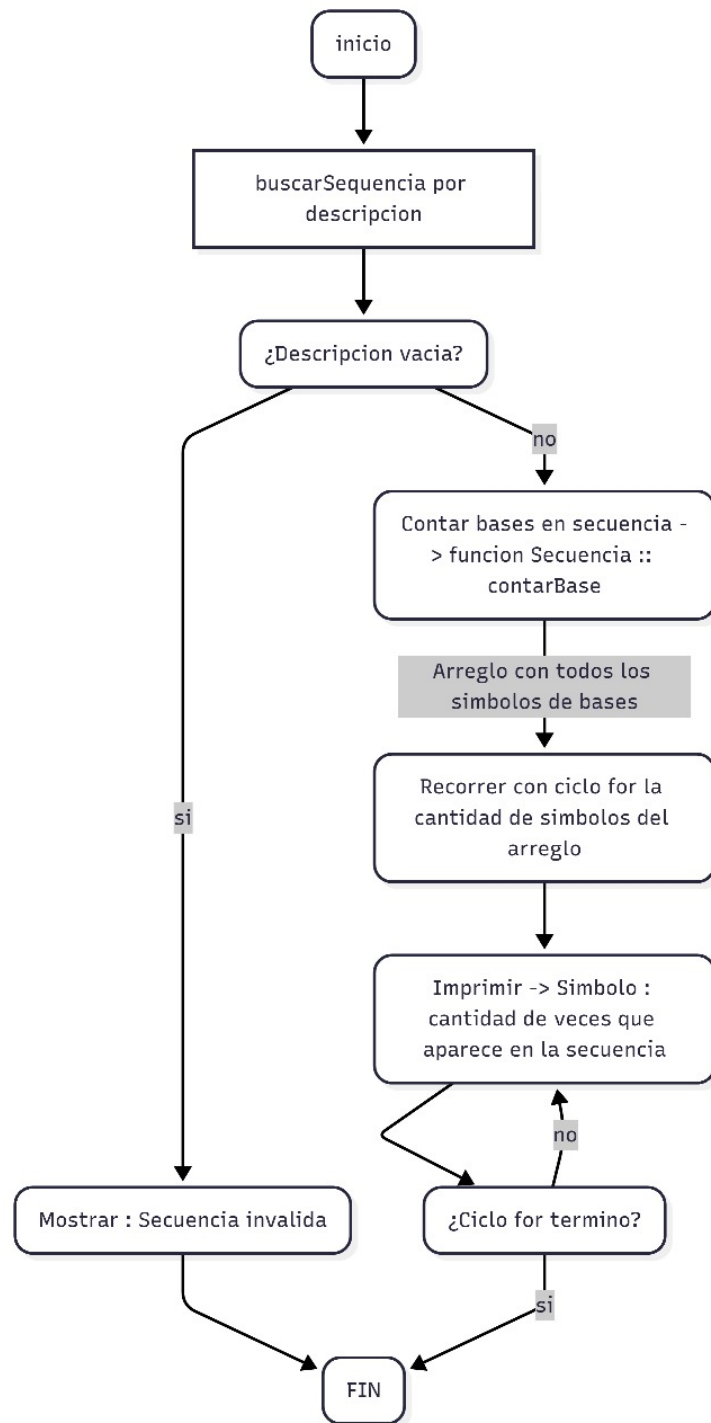


Figura 10. Diagrama de flujo – función buscarSecuencia

Descripción: Busca la secuencia en la lista de secuencias dada una descripción

Entrega 2

Función codificar:

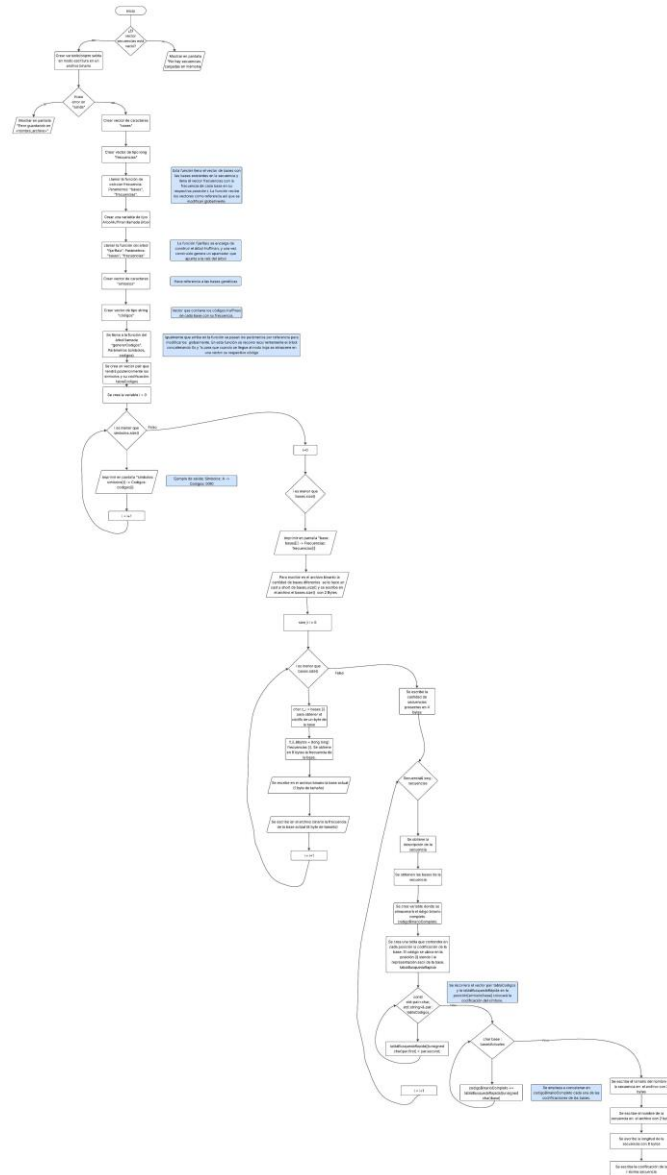


Figura 11. Diagrama de flujo – función codificar

Debido a la calidad de la imagen para poder observar mejor el diagrama se recomienda dirigirse a la sección de anexos.

Función decodificar:



Figura 12. Diagrama de flujo – función decodificar

Entrega 3

Función ruta_mas_corta:

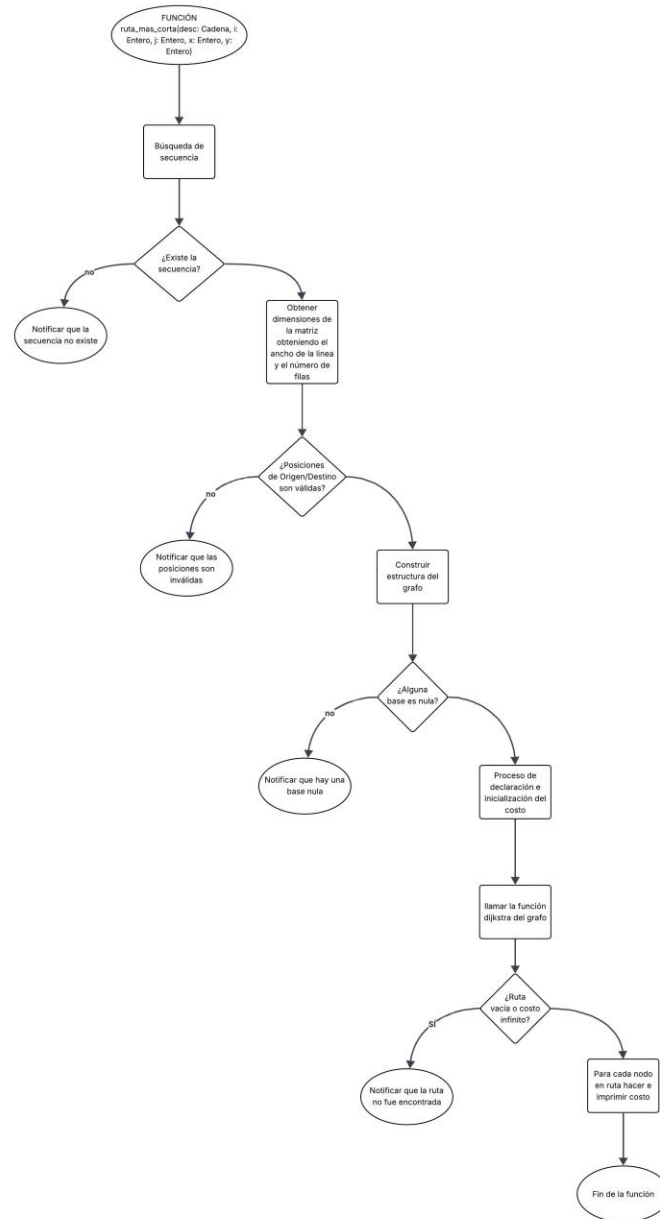


Figura 13. Diagrama de flujo – función ruta_mas_corta

Función base_remota:

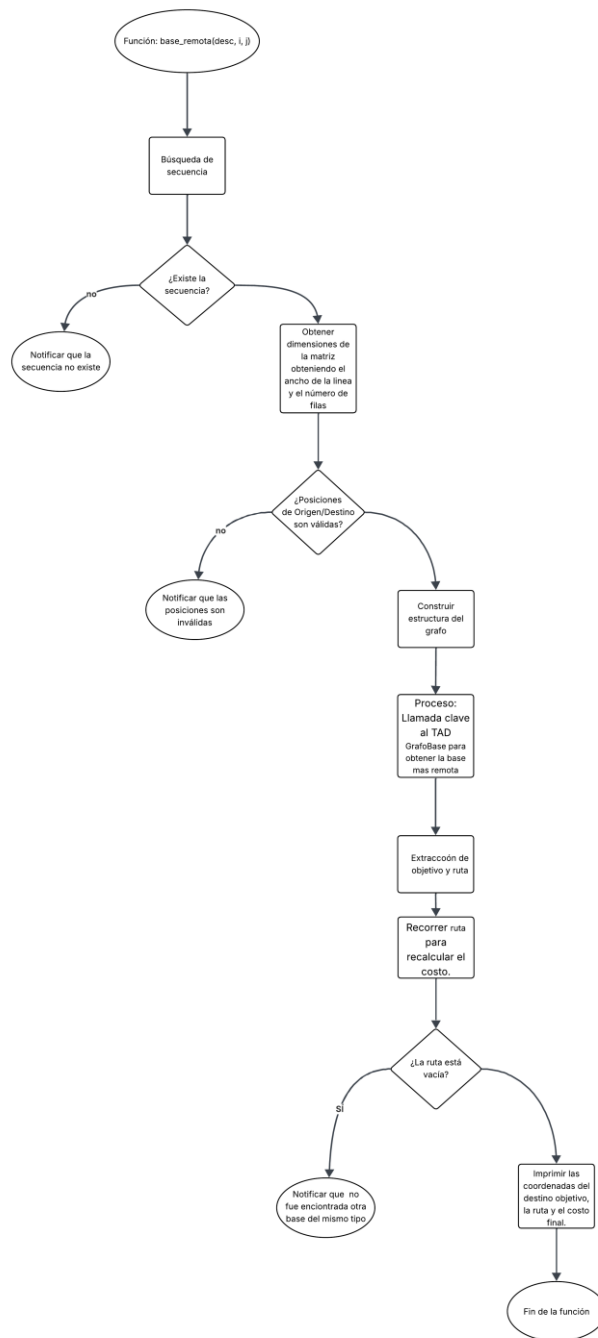


Figura 14. Diagrama de flujo – base_remota

Plan de pruebas

Entrega 1

Comando: enmascarar subsecuencia

Casos de prueba:

ID	Descripción	Entrada	Resultado esperado
PR-01	No hay secuencias cargadas	Enmascarar CATC	No hay secuencias cargadas en memoria.
PR-02	Subsec. no encontrada	Enmascarar CATC	La subsecuencia dada no existe dentro de las secuencias cargadas en memoria, por tanto, no se enmascara nada.
PR-03	Subsec. encontrada 1 vez	Enmascarar ANADD	1 subsecuencia ha sido enmascarada dentro de las secuencias cargadas en memoria.
PR-04	Subsec. repetida varias veces	Enmascarar ATT	n subsecuencias han sido enmascaradas dentro de las secuencias cargadas en memoria.

Comprobación fotográfica

```
$ enmascarar CATC
[OK] comando enmascarar reconocido con subsecuencia: CATC
No hay secuencias cargadas en memoria.
$ cargar secuencias.fasta
[OK] comando cargar reconocido con archivo: secuencias.fasta.
3 secuencias cargadas correctamente desde secuencias.fasta.
$ enmascarar CATC
[OK] comando enmascarar reconocido con subsecuencia: CATC
La subsecuencia dada no existe dentro de las secuencias cargadas en memoria, por tanto no se enmascara nada.
$ enmascarar ANADD
[OK] comando enmascarar reconocido con subsecuencia: ANADD
1 subsecuencias han sido enmascaradas dentro de las secuencias cargadas en memoria.
$ enmascarar ATT
[OK] comando enmascarar reconocido con subsecuencia: ATT
3 subsecuencias han sido enmascaradas dentro de las secuencias cargadas en memoria.
$
```

Entrega 2

Comando: decodificar

Caso	Objetivo	Resultado Esperado	Evidencia fotográfica
1. Archivo válido con 1 secuencia	Verificar que se decodifica correctamente un archivo simple.	Muestra el mensaje: Secuencias decodificadas desde uno.fabin y cargadas en memoria.	<pre>\$ decodificar uno.fabin Secuencias decodificadas desde uno.fabin y cargadas en memoria. \$ listar_secuencias Hay 1 secuencias cargadas en memoria: Secuencia seq1 contiene 155 bases. \$</pre>
2. Archivo válido con varias secuencias	Debe confirmar que puede reconstruir múltiples secuencias en memoria.	Mensaje: Secuencias decodificadas desde yeast.fabin y cargadas en memoria.	<pre>\$ decodificar yeast.fabin Secuencias decodificadas desde yeast.fabin y cargadas en memoria \$ listar_secuencias Hay 17 secuencias cargadas en memoria: Secuencia I contiene 230203 bases. Secuencia II contiene 813139 bases. Secuencia III contiene 316613 bases. Secuencia IV contiene 1531929 bases. Secuencia V contiene 576869 bases. Secuencia VI contiene 278148 bases. Secuencia VII contiene 1090937 bases. Secuencia VIII contiene 562639 bases. Secuencia IX contiene 439885 bases. Secuencia X contiene 745044 bases. Secuencia XI contiene 664445 bases. Secuencia XII contiene 1078173 bases. Secuencia XIII contiene 924430 bases. Secuencia XIV contiene 784328 bases. Secuencia XV contiene 1091284 bases. Secuencia XVI contiene 948861 bases. Secuencia Mit contiene 85779 bases.</pre>
3. Archivo inexistente	Verificar manejo de errores cuando el archivo no existe.	Mensaje: “No se pudo abrir o leer el archivo inexistente.fabin”	<pre>\$ decodificar inexistente.fabin No se pudo abrir o leer el archivo inexistente.fabin</pre>
4. Archivo vacío	Comprobar que maneja correctamente un archivo sin contenido.	Mensaje: Archivo incompleto o corrupto (f_i)	<pre>\$ decodificar vacio.fabin Error: Archivo incompleto o corrupto (f_i).</pre>
5. Archivo dañado (estructura incompleta)	Verificar robustez ante un archivo binario cortado o incompleto.	Mensaje: Archivo incompleto o corrupto (f_i)	<pre>\$ decodificar corrupto.fabin Error: Archivo incompleto o corrupto (f_i).</pre>

6. Comparación de consistencia	de Verificar que— codificar y luego decodificar dan el mismo resultado.	Las secuencias deben tener las mismas descripciones y total de bases (completas o mínimas) que se habían guardado en el archivo creado anteriormente	<pre> Sistema FASTA interactivo - Entrega 1 \$ cargar secuencias.fasta 3 secuencias cargadas correctamente desde secuencias.fasta. \$ listar_secuencias Hay 3 secuencias cargadas en memoria: Secuencia Full_SEQUENCE contiene 155 bases. Secuencia Incomplete_sequence contiene 252 bases. Secuencia seq3 contiene al menos 117 bases. \$ codificar seq.fabin Secuencias codificadas y guardadas en el archivo binario: seq.fabin. \$ salir C:\Users\rcana\Downloads\segundaEntregaFinal>.\prueba Sistema FASTA interactivo - Entrega 1 \$ listar_secuencias No hay secuencias cargadas en memoria. \$ decodificar seq.fabin Secuencias decodificadas desde seq.fabin y cargadas en memoria. \$ listar_secuencias Hay 3 secuencias cargadas en memoria: Secuencia Full_SEQUENCE contiene 155 bases. Secuencia Incomplete_sequence contiene 252 bases. Secuencia seq3 contiene al menos 117 bases. \$ </pre>
--------------------------------	---	--	--

Entrega 3

Comando: base_remota

ID	Descripción del caso	Entrada	Resultado esperado
PR-01	Secuencia no existe	Base_remota "primera" i: 0, j: 0	"La secuencia primera no existe."
PR-02	Posición de base inválida (fila fuera de rango)	Base_remota "seq1", i: -1, j: 0	"La base en la posición [-1,0] no existe."
PR-03	Posición de base inválida (columna fuera de rango)	Base_remota "seq1", i: 0, j: 100	"La base en la posición [0,100] no existe."
PR-04	Ruta vacía (origen = remota)	Base_remota "seq1", i: 0, j: 0	La ruta contiene únicamente la base de origen; costo total = 0.
P-R-05	Ruta con varias bases intermedias	Base_remota "seq1", i: 0, j: 0	Se imprime la secuencia de bases que forman la ruta completa y el costo total calculado.

Comprobación fotográfica

```
Sistema FASTA interactivo - Entrega Final
$ cargar pr.fa
1 secuencia cargada correctamente desde pr.fa.
$ base_remota primera 0 0
La secuencia primera no existe.
$ base_remota seq1 -1 0
La base en la posicion [-1,0] no existe.
$ base_remota seq1 1 100
La base en la posicion [1,100] no existe.
$ base_remota seq1 0 0
Para la secuencia seq1, la base remota esta en [0,2].
Ruta: T -> G -> T -> A -> G -> C -> T
Costo total: 0.59127
$ cargar pruno.fa
1 secuencia cargada correctamente desde pruno.fa.
$ base_remota seq1 0 0
Para la secuencia seq1, la base remota esta en [0,0].
Ruta: T
Costo: 0
```